Studio Nx

JS 77 — 페이지 시작 코드

2026-10-12

JS 77 — 페이지 시작 코드 (`mypage.js` 맨 아래)

항목	내용
파일	`studionx/frontend/static/js/pages/mypage.js`
작업	P1-17 마이페이지 JS: 로그인 확인, 대시보드 받기, 네 곳 채우기를 한 흐름으로 연결 (FR-127~FR-133)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	76 `initLogoutButton()` · 개요 55 (다음)

1. 하는 일

지금까지 만든 함수들(70~76)을 페이지가 열릴 때 순서대로 부르는 곳입니다. 템플릿에 TODO 두 줄로 남아 있던 부분을 채웠습니다.

순서	코드	하는 일
1	`await includePartials()`	공통 머리글과 바닥글을 끼웁니다 (기존)
2	`initAuthState()`	머리글을 로그인 상태에 맞춥니다 (기존)
3	`initDesktopOnlyLinks(#recent-list)`, `initDesktopOnlyLinks(#ready-banner)`	작은 화면에서 시퀀서 링크를 누르면 안내 창을 띄웁니다 (FR-129)
4	`initLogoutButton()`	로그아웃 버튼을 연결합니다 (76번)
5	`await requireLogin()`	로그인하지 않았으면 `home.html#login?next=…`으로 보냅니다
6	`await loadDashboard()`	`GET /me/dashboard`를 받습니다 (70번). 실패하면 오류 화면으로 옮겨 가고 `null`을 돌려줍니다
7	렌더 함수 네 개	`renderReadyBanner`, `renderProfile`, `renderUsage`, `renderRecentProjects` (71~75번)

2. 코드

```
await includePartials();

initAuthState();
// 작은 화면에서 '시퀀서 열기' 링크를 누르면 안내 (FR-129)
initDesktopOnlyLinks(document.getElementById('recent-list'));
initDesktopOnlyLinks(document.getElementById('ready-banner'));
initLogoutButton();   // 대시보드를 기다리는 동안에도 누를 수 있게 먼저 연결
if (await requireLogin()) {
  const data = await loadDashboard();
  if (data) {
    renderReadyBanner(data.ready_projects);
    renderProfile(data.user);
    renderUsage(data.storage, data.ai);
    renderRecentProjects(data.recent_projects);
  }
}
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다)

```
if (await requireLogin()) {
  const data = await loadDashboard();
  if (data) {
    const { ready_projects, user, storage, ai, recent_projects } = data;
    renderReadyBanner(ready_projects);
    renderProfile(user);
    renderUsage(storage, ai);
    renderRecentProjects(recent_projects);
  }
}
```

최신 문법	실제 코드	뜻
`const { user, storage } = data`	`data.user`, `data.storage`	객체에서 필요한 칸만 한 번에 꺼냅니다 (구조 분해)

3. 흐름

1. 머리글과 바닥글이 들어온 뒤에 링크와 버튼을 연결합니다. 요소가 있어야 찾을 수 있기 때문입니다.
2. `requireLogin()`이 `false`이면 이미 로그인 화면으로 이동하는 중이므로 아무것도 그리지 않습니다.
3. `loadDashboard()`가 `null`이면 오류 화면으로 이동하는 중이므로 그리지 않습니다.
4. 데이터가 오면 네 곳을 채웁니다. 화면의 예시 값은 이때 실제 값으로 바뀝니다.

4. 왜 이렇게

선택	이유
링크 연결 (`initDesktopOnlyLinks`)을 데이터보다 먼저	이 함수는 상자에 클릭 처리기를 하나만 걸고 눌린 곳을 판별하므로, 목록이 나중에 그려져도 동작합니다. 그래서 데이터를 기다릴 필요가 없습니다
`#ready-banner`에도 연결	배너의 `Open in studio`도 작은 화면에서는 안내 창을 띄워야 합니다. 제목은 75번이 넣어 둔 `data-title`을 읽습니다
로그아웃 버튼을 로그인 확인보다 먼저	대시보드가 느리거나 실패해도 버튼은 눌려야 합니다. 회원 정보 화면은 뒤에 두지만, 여기서는 기다리는 시간이 길 수 있어 앞으로 옮겼습니다
`if (await requireLogin())`	회원 정보 화면(`userinfo.js`)과 같은 형태입니다
`data`가 `null`이면 그리지 않음	`loadDashboard`가 이미 오류 화면으로 보냈기 때문입니다
네 함수를 한 번에 부르고 따로 `try`를 두지 않음	각 함수가 칸이 없어도 멈추지 않도록 이미 방어되어 있습니다

5. 시험 (실제 myPage.html, 헤드리스 Chrome, 5개)

시험	결과
로그아웃 상태로 열기	`home.html#login?next=…myPage.html`로 이동합니다
로그인 상태로 열기	닉네임, 이메일, 배너의 `Lo-fi Sketch`, 최근 프로젝트 줄이 실제 API 값으로 채워지고 로그아웃 버튼이 연결됩니다
폭 1024에서 최근 프로젝트 링크 누르기	페이지가 그대로이고 안내 창이 뜹니다

시험	결과
폭 1024에서 배너의 `Open in studio` 누르기	안내 창의 이름이 `Lo-fi Sketch`입니다
대시보드 API가 500을 돌려줄 때	`errors/500.html`로 이동합니다

5개 모두 통과했습니다. 70~76번 시험도 다시 돌렸습니다. 이제 로그인이 필요한 화면이어서, 시험 쪽에 로그인 단계를 넣었습니다.